You said:

I'm doing a lab assignment on vibe coding using an LLM as a coding assistant

The project is a terminal Blackjack vs Dealer game in Python (**https://github.com/SETAPESU26/18_blackjack**).
Files: main.py (entry point), game.py (round flow, decisions, chip balance),
cards.py (deck operations and hand scoring), requirements.txt.
Instructions:
1. Go to the repo assigned to you
2. Go through the ReadMe file to understand the deliverables
3. Clone or Fork the repo
4. Run the Python based code (A game/ A Puzzle)
5. Record 10 seconds of video before making any changes.
6. Write prompts and fix the broken code
7. Write prompts to add all the features listed in the ReadMe file of the repo one
by one.
8. Record another 10 seconds of video again after making all the changes.
9. Push the code to your repo.
10.Have Separate Commits for each task
11.Do not raise PR to the main repo (SETAPESU26)

Deliverables:
Upload the following to your Lab repo under Lab-4 folder,

a. Videos (before and after)
b. Updated Code
c. Chat history exported as a doc/pdf
- Explain every change you make and why.

I'll paste the three files below. FIRST, do not change anything. Just:
1. Summarise what each file does.
2. Inspect the hand-value function in cards.py and manually reason through
   hands with one Ace and multiple Aces (A+9, A+9+5, A+A+9).
3. Tell me what incorrect totals the current code would produce, so I can
   reproduce the bug before fixing it.

[paste main.py]

[paste game.py]
[paste cards.py]
[paste requirements.txt]

ChatGPT said:

Understood. I'll **inspect the code only and make no changes** until you explicitly ask me to fix something.

Please paste the contents of the four files, preferably in separate code blocks:

1. `main.py`
2. `game.py`
3. `cards.py`
4. `requirements.txt`

Once you share them, I'll provide the following.

1. Summary of each file

Explain the purpose of each file, its main functions, and how the files work together to run the Blackjack game.

2. Manual inspection of the hand-value function

I'll focus on how the current code calculates the value of Aces, without modifying the function.

Hand	Correct Blackjack value	What I'll check
A + 9	20	Whether the Ace counts as 11
A + 9 + 5	15	Whether the Ace correctly changes from 11 to 1
A + A + 9	21	Whether multiple Aces are handled correctly

3. Identify the existing bug

For each hand, I'll trace the current implementation step by step and report:

- The total the existing code would calculate.
- The correct total under Blackjack rules.
- Why the two totals differ, if they do.
- The exact test cases you can run to reproduce the bug before fixing it.

Important: I won't assume the function is broken in a particular way before seeing its implementation. The actual incorrect totals depend on the logic in your `cards.py`.

This will give you a clear **before-fix explanation** for your lab report and exported coding-assistant chat history. We'll keep the original code unchanged during this first stage.

You said:

main.py

```
from game import Blackjack

if __name__ == "__main__":
    Blackjack().run()
```

game.py

```
from cards import Deck, hand_value


class Blackjack:
    def __init__(self):
        self.chips = 100

    def show(self, player, dealer, hide=True):
        shown_dealer = ["??"] if hide else [f"{r}{s}"
        print("Dealer:", " ".join(shown_dealer))
        print("Player:", " ".join(f"{r}{s}" for r, s i
              "=", hand_value(player))

    def round(self):
        deck = Deck()
        player = [deck.draw(), deck.draw()]
        dealer = [deck.draw(), deck.draw()]
        self.show(player, dealer)

        while hand_value(player) < 21:
            key = input("[h]it [s]tand [q]uit: ").stri
            if key == "q":
                return False
            if key == "s":
                break
            if key == "h":
                player.append(deck.draw())
                self.show(player, dealer)
                if hand_value(player) > 21:
                    print("Bust.")
                    return True
        while hand_value(dealer) < 17:
            dealer.append(deck.draw())

        self.show(player, dealer, hide=False)
```

```
        pv, dv = hand_value(player), hand_value(dealer
        if dv > 21 or pv > dv:
            self.chips += 10
            print("Player wins.")
        elif pv < dv:
            self.chips -= 10
            print("Dealer wins.")
        else:
            print("Push.")
        return True

    def run(self):
        print("Blackjack — starting chips:", self.chip
        while self.chips > 0:
            if not self.round():
                return
            if input("Play again? [y/n]: ").strip().lo
                return
```

cards.py

```
import random

RANKS = ["2", "3", "4", "5", "6", "7", "8", "9", "10",
SUITS = ["C", "D", "H", "S"]


class Deck:
    def __init__(self):
        self.cards = [(rank, suit) for suit in SUITS f
        random.shuffle(self.cards)

    def draw(self):
        return self.cards.pop() if self.cards else Non


def hand_value(hand):

    value = sum(11 if rank == "A" else 10 if rank in {
                for rank, _ in hand)
    return value
```

nothing in requirements.txt

ChatGPT said:

Stage 1: Understand the Blackjack project and reproduce the bug

We will **not change any code yet**. First, let's understand how the project works, inspect the existing Ace-scoring logic, and identify the incorrect totals you can demonstrate in your 10-second before video.

1. What each file does

`main.py` — Entry point

```
from game import Blackjack

if __name__ == "__main__":
    Blackjack().run()
```

- Imports the `Blackjack` class from `game.py`.
- Creates a new Blackjack game.
- Calls `run()` to start the game when `main.py` is executed directly.

In simple terms: This file starts the game. It does not contain the game rules.

`game.py` — Game flow and decisions

This file controls the gameplay.

Function	Purpose
`__init__()`	Initializes the player's chip balance to 100.
`show()`	Displays the player's hand and the dealer's visible or hidden cards.
`round()`	Deals cards, accepts hit/stand/quit input, plays the dealer's hand, and determines the winner.
`run()`	Runs successive rounds until the player quits or runs out of chips.

The game imports two components from `cards.py`:

```
from cards import Deck, hand_value
```

- `Deck` supplies shuffled cards.
- `hand_value()` calculates the value of a hand.

The scoring function is particularly important because the game uses its result to determine whether a player busts, whether the dealer draws more cards, and who wins.

`cards.py` — **Deck management and hand scoring**

This file defines the card data and the scoring logic.

- `RANKS` : Card ranks from 2 through 10, plus J, Q, K and A.
- `SUITS` : Clubs, Diamonds, Hearts and Spades.
- `Deck.__init__()` : Creates a standard 52-card deck and shuffles it.
- `Deck.draw()` : Removes and returns one card from the deck.
- `hand_value(hand)` : Calculates the numerical value of a hand.

The problem is in the final function: **every Ace is always counted as 11**, regardless of whether that causes the hand to exceed 21.

`requirements.txt` — **Dependencies**

You stated that this file is empty. That is fine for the current project because it uses only Python's built-in `random` module and the built-in language features. No third-party packages are required by the code shown.

2. Inspecting the hand-value function

Here is the current implementation, unchanged:

```
def hand_value(hand):

    value = sum(11 if rank == "A" else 10 if rank in {"J", "Q", "K"} else int(rank)
                for rank, _ in hand)
    return value
```

Let's break down what it does.

1. It loops through every card in the hand.
2. It assigns 11 points to every Ace.
3. It assigns 10 points to J, Q and K.
4. It converts numeric ranks such as `"9"` and `"5"` into integers.
5. It adds all the points together and returns the result.

The function does not check whether the total exceeds 21. It also does not adjust any Ace from 11 points to 1 point.

In Blackjack, an Ace can count as **11 or 1**, depending on which value avoids busting.

3. Manually tracing the three test hands

A + 9

Correct total: 20

Current calculation: 11 + 9 = **20**

Result: Correct. The Ace can count as 11 here.

A + 9 + 5

Correct total: 15

Current calculation: 11 + 9 + 5 = **25**

Result: Incorrect. The function returns 25 instead of 15.

The Ace should change from 11 to 1, reducing the total by 10.

A + A + 9

Correct total: 21

Current calculation: 11 + 11 + 9 = **31**

Result: Incorrect. The function returns 31 instead of 21.

One Ace should count as 11 and the other as 1, giving 11 + 1 + 9 = 21.

4. Summary of the bug

Hand	Current output	Correct output	Difference
A + 9	20	20	None
A + 9 + 5	25	15	+10
A + A + 9	31	21	+10

The current code does not always produce an incorrect total. It works when counting every Ace as 11 is valid, but it fails when an Ace needs to count as 1.

This is a useful bug to demonstrate because the game can make incorrect decisions based on the total:

- With A + 9 + 5, the player is incorrectly treated as having busted.
- With A + A + 9, the player is incorrectly treated as having busted instead of reaching 21.

The same scoring issue can also affect dealer decisions and the winner calculation.

5. How to reproduce the bug before fixing it

You can test the existing function directly without changing any project files. Open a terminal in the repository folder and run:

```
python
```

Then enter:

```
from cards import hand_value

print(hand_value([("A", "S"), ("9", "H")]))
print(hand_value([("A", "S"), ("9", "H"), ("5", "D")]))
print(hand_value([("A", "S"), ("A", "H"), ("9", "D")]))
```

Expected output from the **current, unfixed code**:

```
20
25
31
```

These outputs establish the original behaviour. You can use them as evidence in your before-fix documentation.

To exit the Python interpreter, type:

```
exit()
```

6. What to capture in your before video

For your 10-second recording, I recommend showing the existing game running before you make any changes.

1. Open the terminal in the repository.
2. Run `python main.py` .
3. Show the initial Blackjack screen and the player's and dealer's cards.
4. Keep the recording short, as required by your assignment.

The direct scoring tests above are useful additional evidence for your chat-history document, even if you do not include them in the 10-second gameplay video.

Current status: No files have been modified. The confirmed scoring bug is that Aces are always assigned 11 points and are never adjusted to 1. The next stage, when you are ready, can be to write a focused LLM prompt to fix this function, test the three cases again, and commit that change separately.

You said:

Okay... I ran the game.. in the game the hand
JC 2C AH showed 23 and gave bust, but it should be 13. When I called
hand_value on A+9+5 it gave me 25, so it matches what you predicted.

Can you fix hand_value in cards.py? An Ace should count as 11 unless that would
push the value more than 21 else it should be counted as 1.
Please only change that one function...keep it simple, and explain how it works
in plain words so i can understand it completely.

ChatGPT said:

Let's fix only `hand_value()` in `cards.py`, keeping the rest of your Blackjack project unchanged.

Your example, `JC 2C AH`, confirms the problem:

- Jack of Clubs (`JC`) = 10
- 2 of Clubs (`2C`) = 2
- Ace of Hearts (`AH`) = 11 initially
- The current code calculates $10 + 2 + 11 = 23$ and incorrectly declares a bust.
- The correct value is $10 + 2 + 1 = 13$, because counting the Ace as 11 would exceed 21.

The simplest fix is to **count every Ace as 11 initially, then subtract 10 for each Ace that needs to count as 1 to avoid busting.**

1. Replace only this function

In `cards.py`, replace the existing `hand_value()` function with the following code. Leave everything else exactly as it is.

Writing

```python
def hand_value(hand):
    value = sum(11 if rank == "A" else 10 if rank in {"J", "Q", "K"} else int(rank)
                for rank, _ in hand)

    aces = sum(1 for rank, _ in hand if rank == "A")

    while value > 21 and aces > 0:
        value -= 10
        aces -= 1

    return value
```

2. How does this work?

Let's understand each part in plain English.

Step 1: Calculate the initial value

```
value = sum(11 if rank == "A" else 10 if rank in {"J", "Q", "K"} else int(rank)
            for rank, _ in hand)
```

This is your original calculation. It assigns 11 to every Ace, 10 to face cards, and their numerical values to other cards.

For `JC 2C AH`, the initial value is 23.

Step 2: Count the Aces

```
aces = sum(1 for rank, _ in hand if rank == "A")
```

This counts how many Aces are in the hand.

- No Aces → `aces = 0`
- One Ace → `aces = 1`
- Two Aces → `aces = 2`

We need this count because each Ace can reduce the total by 10 points, changing its value from 11 to 1.

Step 3: Adjust the total if necessary

```
while value > 21 and aces > 0:
    value -= 10
    aces -= 1
```

This is the key part of the fix.

- `value > 21` checks whether the hand has busted.
- `aces > 0` checks whether there is still an Ace available to adjust.
- `value -= 10` changes one Ace from 11 to 1.
- `aces -= 1` records that this Ace has been adjusted.

The `while` loop repeats if the hand still exceeds 21 and another Ace is available. If the total is already 21 or lower, no adjustment is needed.

Step 4: Return the final value

```
return value
```

The function returns the adjusted total to the game. The existing game logic can then use it to determine whether the player has busted.

3. Test the fix

After saving `cards.py`, run Python in your project directory:

```
from cards import hand_value

print(hand_value([("J", "C"), ("2", "C"), ("A", "H")]))
print(hand_value([("A", "S"), ("9", "H")]))
print(hand_value([("A", "S"), ("9", "H"), ("5", "D")]))
print(hand_value([("A", "S"), ("A", "H"), ("9", "D")]))
```

Expected output:

J + 2 + A Before: 23	→	**13** Correct
A + 9 Before: 20	→	**20** Correct
A + 9 + 5 Before: 25	→	**15** Correct
A + A + 9 Before: 31	→	**21** Correct

4. Why this solution works for multiple Aces

Consider `A + A + 9`.

1. Initially, the total is $11 + 11 + 9 = 31$.
2. The total exceeds 21, so the loop subtracts 10. The total becomes 21, and one Ace remains available.
3. Since 21 is not greater than 21, the loop stops.
4. The function returns 21.

It also handles hands with no Aces: if the total exceeds 21, the loop cannot adjust anything because `aces` is zero.

What changed? Only the `hand_value()` function. The deck, game flow, chip balance, and user input logic remain untouched.

For your lab report, you can explain the change as: *"I modified the hand-value function to count Aces as 11 initially and dynamically reduce their value to 1 when necessary to prevent a bust. This fixes incorrect scoring for hands containing one or multiple Aces."*

You said:

Fix is working..... J+2+A =13, scoring works now.
Next I want to clean up how a round finishes in game.py..now a two-card
21 is not treated as a blackjack, and I'm not sure every
ending is handled properly.
can you see if these are working... blackjack for the player
or the dealer, player bust, dealer bust, the dealer drawing until 17 or more,
and ties ? also make sure the player can't hit or stand once the round
is already over.
can you keep the changes small, explain what was changed and why so that i can understand completely.

ChatGPT said:

Great! Since `hand_value()` is now working, let's focus on `game.py` and make the round endings reliable without rewriting the whole game.

There are a few things to address in the current implementation:

- **Blackjack:** A two-card 21 is currently treated like an ordinary 21. The game doesn't explicitly distinguish a natural Blackjack from a 21 reached with three or more cards.
- **Player bust:** The existing code handles this, but it returns immediately without revealing the dealer's hand.
- **Dealer bust:** The existing winner calculation handles this.
- **Dealer drawing:** The dealer correctly draws while the total is below 17.
- **Ties:** Equal totals are currently treated as a push, which is correct for ordinary ties, but natural Blackjack needs explicit handling.
- **Round over:** The current code already exits the player's input loop in several cases, but we can make the round's ending decisions clearer so the player cannot act after the round has finished.

One important Blackjack rule: a natural Blackjack is an Ace plus a 10-value card in the initial two cards. It normally beats a non-Blackjack 21, while two natural Blackjacks push. We'll implement that distinction.

1. Make a small change to `game.py`

Replace **only the existing `round()` method** inside the `Blackjack` class with the version below. Keep `__init__()`, `show()`, and `run()` unchanged.

Writing

```python
def round(self):
    deck = Deck()
    player = [deck.draw(), deck.draw()]
    dealer = [deck.draw(), deck.draw()]
    self.show(player, dealer)

    # Check for Blackjack on the initial two cards.
    player_bj = len(player) == 2 and hand_value(player) == 21
    dealer_bj = len(dealer) == 2 and hand_value(dealer) == 21

    if player_bj or dealer_bj:
        self.show(player, dealer, hide=False)

        if player_bj and dealer_bj:
            print("Push. Both have Blackjack.")
        elif player_bj:
            self.chips += 10
            print("Blackjack! Player wins.")
        else:
            self.chips -= 10
            print("Dealer has Blackjack. Dealer wins.")

        return True

    # Player's turn.
    while hand_value(player) < 21:
        key = input("[h]it [s]tand [q]uit: ").strip().lower()

        if key == "q":
            return False
        if key == "s":
            break
        if key == "h":
            player.append(deck.draw())
            self.show(player, dealer)

            if hand_value(player) > 21:
```

```
                self.show(player, dealer, hide=False)
                print("Bust.")
                return True

        # Dealer draws until reaching at least 17.
        while hand_value(dealer) < 17:
            dealer.append(deck.draw())

        self.show(player, dealer, hide=False)
        pv, dv = hand_value(player), hand_value(dealer)

        if dv > 21 or pv > dv:
            self.chips += 10
            print("Player wins.")
        elif pv < dv:
            self.chips -= 10
            print("Dealer wins.")
        else:
            print("Push.")

        return True
```

2. What changed, and why?

Change 1: Detect Blackjack immediately

```
player_bj = len(player) == 2 and hand_value(player) == 21
dealer_bj = len(dealer) == 2 and hand_value(dealer) == 21
```

A total of 21 alone is not enough to qualify as Blackjack. The hand must contain exactly two cards.

For example:

- `A + K` → natural Blackjack.
- `A + 9 + A` → 21, but not natural Blackjack.

If either player or dealer has Blackjack, the game reveals both hands, determines the result, and returns from `round()`. No further hit or stand input is accepted.

Change 2: Handle all natural Blackjack outcomes

Player	Dealer	Result
Blackjack	No Blackjack	Player wins
No Blackjack	Blackjack	Dealer wins
Blackjack	Blackjack	Push (tie)

This also ensures a dealer's natural Blackjack beats a player's 21 made with three or more cards.

Change 3: Make the bust ending clearer

```
if hand_value(player) > 21:
    self.show(player, dealer, hide=False)
    print("Bust.")
    return True
```

When the player busts, the game reveals the dealer's hand and ends the round immediately. Returning from the method means the player cannot continue hitting or standing in that round.

Change 4: Keep dealer drawing and ordinary results

The existing dealer rule remains unchanged:

```
while hand_value(dealer) < 17:
    dealer.append(deck.draw())
```

The dealer draws below 17 and stops at 17 or above.

The final comparisons also remain the same:

- Dealer busts → player wins.
- Player's total is higher → player wins.
- Dealer's total is higher → dealer wins.
- Equal totals → push.

The existing chip changes of `+10` for a win and `-10` for a loss are preserved. This update does not introduce a special Blackjack payout.

3. Test checklist

After saving `game.py`, run `python main.py` and verify the following cases. Since the deck is shuffled randomly, you may need multiple rounds to encounter particular hands.

Verification checklist 0 of 8 checked

- [ ] **Player has a natural Blackjack**
  An initial two-card 21 wins against a dealer without Blackjack.
- [ ] **Dealer has a natural Blackjack**
  The round ends immediately and the dealer wins unless the player also has Blackjack.
- [ ] **Both have Blackjack**
  The result is a push, with no chip change.
- [ ] **Player busts**
  The dealer's hand is revealed and no more player input is requested.
- [ ] **Dealer busts**
  The player wins when the dealer exceeds 21.
- [ ] **Dealer draws below 17**
  The dealer keeps drawing until reaching at least 17.
- [ ] **Equal ordinary totals**
  The round ends in a push.
- [ ] **Player reaches 21 after hitting**
  The player's turn ends automatically; no further hit or stand prompt appears.

4. Suggested Git commit

Once you have tested the changes, make a separate commit for this task, as required by your lab instructions.

```
git add game.py
git commit -m "Fix Blackjack round endings"
```

Lab explanation: *"I updated the round method to detect natural Blackjack for both player and dealer, resolve both-Blackjack ties, reveal the dealer's hand after a player bust, and end each round immediately when its outcome is determined. The dealer's draw-until-17 rule and existing chip-scoring rules are preserved."*

One final distinction: the `return` statements end the current round, while `run()` may still ask whether you want to play another round. That is expected behavior, not an extra hit or stand opportunity.

You said:

Task2 is done...next I want chips to carry over
between rounds and let me place bets...before each round, ask for a bet and keep asking until it's valid....reject

empty input, text, zero, negatives, decimals, and anything over my current
chips.
Make chips persist across rounds and add a simple wager mechanism with validation. A round must settle exactly once.Show my chips at the start of each round and after the result...if I hit 0 chips, show a game over message instead of asking for another bet...keep it in memory only, change as little as possible, and explain what you changed in simple words so that i can understand everything.

ChatGPT said:

We'll make this change entirely in memory for now, without touching GitHub or creating any commits.

The key idea is to replace the fixed `+10 / -10` chip changes with a **wager chosen before each round**. We'll validate the bet before dealing cards, settle it once when the round ends, and show the remaining chips.

We'll keep the changes small by modifying only `game.py`. Your existing `cards.py` scoring fix and the rest of the project will stay untouched.

1. How the new betting system will work

- Each round starts by displaying the current chip balance.
- The player enters a whole-number bet between `1` and their available chips.
- Empty input, text, zero, negative numbers, decimals, and bets larger than the balance are rejected.
- The game deals cards only after a valid bet is entered.
- A win adds the bet; a loss subtracts the bet; a push returns the bet unchanged.
- A natural Blackjack follows the same wager settlement rule for now—no special payout.
- If the balance reaches zero, the game displays **Game Over** and does not ask for another bet.
- Quitting during the player's turn cancels that round without settling a wager.

2. Updated `game.py`

Replace the existing `round()` and `run()` methods inside the `Blackjack` class with these versions. Leave `__init__()`, `show()`, and `cards.py` unchanged.

Writing

```python
def round(self):
```

```
print("\nChips:", self.chips)

# Ask for a valid bet.
while True:
    raw_bet = input("Place your bet: ").strip()

    try:
        bet = int(raw_bet)
    except ValueError:
        print("Invalid bet. Enter a whole number.")
        continue

    if bet <= 0:
        print("Bet must be greater than zero.")
    elif bet > self.chips:
        print("You cannot bet more chips than you have.")
    else:
        break

deck = Deck()
player = [deck.draw(), deck.draw()]
dealer = [deck.draw(), deck.draw()]
self.show(player, dealer)

# Settle the wager once when the round ends.
def finish(result, message):
    if result == "win":
        self.chips += bet
    elif result == "loss":
        self.chips -= bet

    print(message)
    print("Chips after round:", self.chips)
    return True

# Check for Blackjack on the initial two cards.
player_bj = len(player) == 2 and hand_value(player) == 21
dealer_bj = len(dealer) == 2 and hand_value(dealer) == 21

if player_bj or dealer_bj:
    self.show(player, dealer, hide=False)
```

```
        if player_bj and dealer_bj:
            return finish("push", "Push. Both have Blackjack.")
        elif player_bj:
            return finish("win", "Blackjack! Player wins.")
        else:
            return finish("loss", "Dealer has Blackjack. Dealer wins.")

    # Player's turn.
    while hand_value(player) < 21:
        key = input("[h]it [s]tand [q]uit: ").strip().lower()

        if key == "q":
            return False
        if key == "s":
            break
        if key == "h":
            player.append(deck.draw())
            self.show(player, dealer)

            if hand_value(player) > 21:
                self.show(player, dealer, hide=False)
                return finish("loss", "Bust.")

    # Dealer draws until reaching at least 17.
    while hand_value(dealer) < 17:
        dealer.append(deck.draw())

    self.show(player, dealer, hide=False)
    pv, dv = hand_value(player), hand_value(dealer)

    if dv > 21 or pv > dv:
        return finish("win", "Player wins.")
    elif pv < dv:
        return finish("loss", "Dealer wins.")
    else:
        return finish("push", "Push.")



def run(self):
    print("Blackjack — starting chips:", self.chips)
```

```
        while self.chips > 0:
            if not self.round():
                return

            if self.chips == 0:
                print("Game Over! You have no chips left.")
                return

            if input("Play again? [y/n]: ").strip().lower() != "y":
                return

        print("Game Over! You have no chips left.")
```

3. Understand the changes

A. Bet validation

```
raw_bet = input("Place your bet: ").strip()

try:
    bet = int(raw_bet)
except ValueError:
    print("Invalid bet. Enter a whole number.")
    continue
```

`strip()` removes spaces around the input. `int()` accepts whole numbers such as `5`, but rejects empty input, text such as `hello`, and decimal input such as `2.5`.

The `continue` statement sends the player back to the bet prompt if conversion fails.

Next, we check the bet's range:

```
if bet <= 0:
    print("Bet must be greater than zero.")
elif bet > self.chips:
    print("You cannot bet more chips than you have.")
else:
    break
```

- `bet <= 0` rejects zero and negative bets.

- `bet > self.chips` rejects bets exceeding the current balance.
- `break` exits the validation loop only when the bet is valid.

No cards are dealt until validation succeeds.

B. One place to settle the wager

```
def finish(result, message):
    if result == "win":
        self.chips += bet
    elif result == "loss":
        self.chips -= bet

    print(message)
    print("Chips after round:", self.chips)
    return True
```

This small helper keeps settlement consistent:

Round result	Chip change
Win	Add the bet
Loss	Subtract the bet
Push (tie)	No change

For example, if you start with 100 chips and bet 20:

- Win → 120 chips.
- Loss → 80 chips.
- Push → 100 chips.

Notice that we **do not subtract the bet before playing**. We update the balance only when the result is settled, which makes the calculations easier to understand.

Every completed outcome calls `finish()` once and immediately returns from `round()`. Quitting during the player's turn returns without settling the bet.

C. Chips persist between rounds

The balance is stored in `self.chips`, which belongs to the `Blackjack` object. Creating a new deck does not reset that balance.

The next round displays the updated balance and uses it to validate the next bet.

This persistence is **in memory only**: if you exit the program and start it again, the balance resets to 100 because it is initialized in `__init__()`.

D. Game over at zero chips

After a round, `run()` checks:

```
if self.chips == 0:
    print("Game Over! You have no chips left.")
    return
```

This check happens before the question asking whether you want to play again. Therefore, the game does not ask for another bet when your balance reaches zero.

4. Test your changes

Test checklist 0/8 tested

- [ ] **Empty bet**
  Input: (press Enter). Rejected; asks again.
- [ ] **Text bet**
  Input: hello. Rejected; asks again.
- [ ] **Zero or negative**
  Input: 0 or -5. Rejected; asks again.
- [ ] **Decimal bet**
  Input: 2.5. Rejected; asks again.
- [ ] **Bet above balance**
  Input: 101 with 100 chips. Rejected; asks again.
- [ ] **Valid bet**
  Input: 20 with 100 chips. Accepted.
- [ ] **Win, loss, or push**
  Input: Bet 20. Balance becomes 120, 80, or 100 respectively.
- [ ] **Zero balance**
  Input: Lose your last chips. Game Over; no further bet prompt.

Run the game with:

```
python main.py
```

One important detail to verify: The code above settles each completed round once. A player who chooses `q` during their turn exits the game without a chip change, and the unfinished round's wager is not charged.

For your lab explanation, you can write:

"I added a validated betting system that accepts only positive whole-number bets within the player's chip balance. I centralized wager settlement in a helper function so wins add the bet, losses subtract it, and pushes leave the balance unchanged. The updated game displays chips before and after each round and stops when the balance reaches zero. All changes are in memory and no persistent storage was added."

You said:

One change....a natural blackjack should pay 3:2
instead of 1:1. Chips are whole numbers, so tell me how you handle odd bets
. Only change the payout part, and explain it simply.

ChatGPT said:

We'll change only the payout logic inside `game.py`. A natural Blackjack (an Ace plus a 10-value card in the initial two cards) will pay **3:2**, while ordinary wins still pay 1:1.

How odd bets will work

Since chips must be whole numbers, we'll calculate the Blackjack profit using integer division:

```
profit = bet * 3 // 2
```

This rounds down when the result is a half-chip.

Bet	Blackjack profit (3:2)	Total balance change
10	15	+15 chips
5	7	+7 chips
3	4	+4 chips
1	1	+1 chip

For odd bets, this means the half-chip is discarded. For example, a bet of 5 earns 7 chips in profit, rather than 7.5.

Make these two small changes

1. Update the `finish()` helper to accept a custom profit for a win:

Writing

```python
def finish(result, message, profit=None):
    if result == "win":
        self.chips += bet if profit is None else profit
    elif result == "loss":
        self.chips -= bet

    print(message)
    print("Chips after round:", self.chips)
    return True
```

2. Change only the player's natural Blackjack outcome in the initial Blackjack check:

Replace:

```
return finish("win", "Blackjack! Player wins.")
```

with:

```
profit = bet * 3 // 2
return finish("win", "Blackjack! Player wins.", profit)
```

Why this works

- **Natural Blackjack:** earns 3:2 profit, rounded down to a whole number.
- **Ordinary win:** still earns 1:1 profit because `profit` defaults to `None`.
- **Loss:** still loses the original bet.
- **Push:** still leaves the balance unchanged.
- **Both player and dealer have natural Blackjack:** still a push, with no payout.

For example, if you have 100 chips and bet 10, a natural Blackjack gives you **115 chips**. Your original 10-chip bet is already included in your balance, so the game adds only the 15-chip profit.

No other files or game rules need to change.

You said:

task 3 is completed.... last part....Task 4 — Action feedback and robustness
Add concise feedback for actual card draws and round outcomes. Invalid commands and invalid wagers must not change the bankroll. while playing show who drew which card and the new total ...at the end of each round, show the result and how many chips is left.
Bad bets should not change my chips. At "Play again?", only y should continue and n should quit. other entries must ask again...
Deck.draw() returns None when the deck runs out which could crash the game..can you ccheck that.. also, q should quit from any prompt, including the bet prompt.
Keep the changes small, explain them in simple words..

ChatGPT said:

For Task 4, we'll build on your completed betting and Blackjack payout tasks, keeping the changes small and limited to `game.py` and `cards.py` where necessary.

The main things to fix are:

- **Card-draw feedback:** show whether the player or dealer drew a card, which card it was, and the new hand total.
- **Round-end feedback:** clearly show the outcome and remaining chips.
- **Input robustness:** invalid bets and commands must not change the bankroll; `q` should quit at both the bet prompt and the player's turn.
- **Play-again validation:** only `y` continues and `n` quits; anything else prompts again.
- **Deck exhaustion:** `Deck.draw()` currently returns `None` when the deck is empty. We need to handle that safely so the game doesn't try to score `None` as a card.

One detail to preserve: your natural Blackjack pays 3:2, while ordinary wins pay 1:1. We'll keep that behavior and your whole-chip handling for odd bets.

1. Update `game.py`

Replace only the existing `round()` and `run()` methods inside the `Blackjack` class with the versions below. Keep `__init__()`, `show()`, and the rest of the file unchanged.

Writing

```python
def round(self):
    print("\nChips at start:", self.chips)

    # Ask for a valid bet.
```

```
while True:
    raw_bet = input("Place your bet (or q to quit): ").strip().lower()

    if raw_bet == "q":
        return False

    try:
        bet = int(raw_bet)
    except ValueError:
        print("Invalid bet. Enter a whole number.")
        continue

    if bet <= 0:
        print("Bet must be greater than zero.")
    elif bet > self.chips:
        print("You cannot bet more chips than you have.")
    else:
        break

deck = Deck()
player = [deck.draw(), deck.draw()]
dealer = [deck.draw(), deck.draw()]

# Safely handle an exhausted deck.
if None in player or None in dealer:
    print("Deck is empty. Round cancelled; chips unchanged.")
    print("Chips remaining:", self.chips)
    return True

self.show(player, dealer)

# Settle the wager once when the round ends.
def finish(result, message, profit=None):
    if result == "win":
        self.chips += bet if profit is None else profit
    elif result == "loss":
        self.chips -= bet

    print(message)
    print("Chips remaining:", self.chips)
    return True
```

```
# Draw a card and report the new total.
def draw_card(hand, who):
    card = deck.draw()

    if card is None:
        print("Deck is empty. Round cancelled; chips unchanged.")
        print("Chips remaining:", self.chips)
        return False

    hand.append(card)
    rank, suit = card
    print(f"{who} drew {rank}{suit}. New total: {hand_value(hand)}")
    return True

# Check for natural Blackjack.
player_bj = len(player) == 2 and hand_value(player) == 21
dealer_bj = len(dealer) == 2 and hand_value(dealer) == 21

if player_bj or dealer_bj:
    self.show(player, dealer, hide=False)

    if player_bj and dealer_bj:
        return finish("push", "Push. Both have Blackjack.")
    elif player_bj:
        profit = bet * 3 // 2
        return finish("win", "Blackjack! Player wins.", profit)
    else:
        return finish("loss", "Dealer has Blackjack. Dealer wins.")

# Player's turn.
while hand_value(player) < 21:
    key = input("[h]it [s]tand [q]uit: ").strip().lower()

    if key == "q":
        return False
    elif key == "s":
        break
    elif key == "h":
        if not draw_card(player, "Player"):
            return True
```

```
            self.show(player, dealer)

            if hand_value(player) > 21:
                self.show(player, dealer, hide=False)
                return finish("loss", "Player busts.")
        else:
            print("Invalid command. Enter h, s, or q.")

    # Dealer draws until reaching at least 17.
    while hand_value(dealer) < 17:
        if not draw_card(dealer, "Dealer"):
            return True

    self.show(player, dealer, hide=False)
    pv, dv = hand_value(player), hand_value(dealer)

    if dv > 21 or pv > dv:
        return finish("win", "Player wins.")
    elif pv < dv:
        return finish("loss", "Dealer wins.")
    else:
        return finish("push", "Push.")



def run(self):
    print("Blackjack — starting chips:", self.chips)

    while self.chips > 0:
        if not self.round():
            return

        if self.chips == 0:
            print("Game Over! You have no chips left.")
            return

        while True:
            answer = input("Play again? [y/n]: ").strip().lower()

            if answer == "q" or answer == "n":
                return
```

```
            elif answer == "y":
                break
            else:
                print("Invalid choice. Enter y, n, or q.")

    print("Game Over! You have no chips left.")
```

2. What changed and why?

A. Feedback when cards are drawn

Whenever the player hits or the dealer draws, the game now prints the card and the new total.

For example:

```
Player drew 7H. New total: 18
Dealer drew 4S. New total: 16
```

The dealer's hidden card remains hidden during the player's turn, as expected in Blackjack. The initial hands are displayed by the existing `show()` function.

B. Invalid inputs don't affect chips

Invalid bets are rejected before cards are dealt. Invalid commands during the player's turn display an error and repeat the prompt.

The bankroll changes only through `finish()`, which handles completed wins, losses, and pushes. This keeps wager settlement in one place and prevents invalid inputs from changing the balance.

C. `q` quits from every prompt

- At the bet prompt, `q` exits before a round begins.
- At the hit/stand prompt, `q` exits without settling the unfinished round.
- At the play-again prompt, `q` exits the game.

Only `y` starts another round and `n` quits normally. Other entries repeat the prompt.

D. The deck cannot crash the round

Your existing `Deck.draw()` already returns `None` when no cards remain. We don't need to modify `cards.py`.

Instead, the updated `game.py` checks for `None` before trying to append or score a drawn card. If the deck runs out during a round, the round is cancelled and the bankroll remains unchanged.

E. Round outcomes and payouts are preserved

The `finish()` helper prints the result and remaining chips. Ordinary wins still pay 1:1, natural Blackjack still pays 3:2 using your existing whole-chip rule, losses subtract the bet, and pushes leave the balance unchanged.

3. Test Task 4

After saving the file, run:

```
python main.py
```

Check these cases:

Task 4 verification 0 of 7

- [ ] **Invalid bets**
  Try empty input, text, 0, -5, 2.5, and a bet above your balance.
- [ ] **Invalid game command**
  Enter an invalid command such as x. The game should ask again.
- [ ] **Card-draw feedback**
  Hit and check that the drawn card and new total are printed.
- [ ] **Play-again validation**
  Try an invalid answer, then y and n. Invalid answers should repeat the prompt.
- [ ] **Quit from prompts**
  Enter q at the bet prompt and at the hit/stand prompt.
- [ ] **Payouts and bankroll**
  Check ordinary wins, losses, pushes, and natural Blackjack payouts.
- [ ] **Deck exhaustion**
  Confirm that a None draw cancels the round without changing chips.

Note: With a freshly shuffled 52-card deck, exhaustion is unlikely during a single round. The exhaustion checks are still useful defensive programming because `Deck.draw()` explicitly allows `None` .

Once these tests pass, you can make a separate Task 4 commit as required by your lab instructions.